GOSIM

GOSIM Shenzhen 2026

AI 原生嵌入式产品研发：

重塑软硬件研发范式

陈昱衡

ambi.ai AI 研究员 · 嵌入式专家

2026 年 10 月



AI 已经能执行，研发为什么仍要靠人逐步推进？

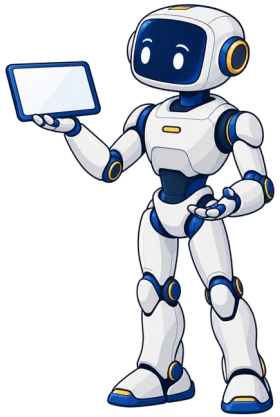


研发是一条连续链路

明确目标 → 拆解任务 → **执行实现** → 检查结果 → 修改推进

过去：分解、执行、检查与交接都由人工推进

现在： AI 接手局部执行，前后仍需要人工衔接



下一步：让 AI 接下整条可验证链路

任务分解 → 方案实现 → 验证反馈 → 修复迭代

检查没通过，就带着具体问题继续修复

人设定目标与边界，持续提需求，保留关键取舍与交付决定

分享路线：让 AI 持续推进产品研发

从工程实践出发，把软件约束、电路知识和实物反馈接入研发流程

01	实践起点	04–07 已有工程积累与黄金参考
02	软件约束	08–23 接口复用、错误处理与资源安全
03	电路约束	24–29 把器件知识写成可复用的规则
04	实物验证	30–33 让 AI 获得观测和实验反馈
05	持续研发	34–50 组织任务、模型协作与路径探索

主线： AI 持续推进，人掌握产品方向与关键取舍

2022 年开始的 Rust 与模块化积累

2022 年起积累模块化与 Rust 实践，接口和资源约定继续复用

2022 年左右

产品模块化
Rust 用于嵌入式产品

基础件逐渐成形

后续产品继续复用
接口与资源约定随模块保留

AI 加入之后

已有结构可作为上下文
减少每次从头猜测

20 余款

基于 Rust 的仪器设备

复用的不只是代码，还有“怎样使用这些代码”的约定

数据范围：本团队的产品实践，不代表行业统计或零故障保证。

早期 AI 实践：从 C 驱动迁移到 Rust

2022 年积累 Rust 与模块化基础，后来让 Cursor 、 Claude Code 接手迁移任务

已有可运行的参考，先验证迁移行为

官方 C 驱动提供参照， Rust 版本要符合团队规范并支持异步

已有官方 C 驱动 → **AI 参与迁移与修改** → **团队需要的 Rust 驱动**

已有行为可以拿来比较

花了很多 token 反复推进

符合团队风格与规范
支持异步，能继续进入产品

迁移后的行为对不对?
要让 AI 一直做下去，先得有办法回答这个问题

早期驱动迁移实践：先对照参考行为，再接入团队工程。

黄金参考：同一条件下比较、修复、重跑

实践起点 / 黄金标准反馈

参考与候选使用可比条件，差异返回 AI，修复后重跑

操作含义对齐，不按源码行号逐行配对

人确定参考与判据

同一任务与输入
可比配置、初始条件

运行并读取实际结果

调试器 / MCP
读出状态、输出与操作记录

AI 定位与修正

哪个操作、哪个对象不同?
修改 Rust，再按原条件重跑

官方 C 参考

初始化后 | 配置后 | 读出后

参考状态 / 结果 | 参考状态 / 结果 | 参考状态 / 结果

同一个观察点，两边比较哪些内容?
状态层 / 过程层 / 结果层

Rust 候选

候选状态 / 结果 | 候选状态 / 结果 | 候选状态 / 结果

比较判据

约定观察项
逐项核对

符合已覆盖判据

保留结果

驱动与运行证据
仍需产品验收

不符合

差异反馈

哪一步、哪个对象、两边差在哪里

方法示意：图中对象仅示意对齐内容，各阶段按具体驱动确定。参考不适用或判据不足时交人判断

黄金参考的多层验证

状态层检查配置，过程层定位分歧，结果层核对实际输出

比较层	参考与候选各取什么		差异怎样帮助定位
① 状态层	寄存器配置 / 状态 约定内存区域的快照	→	哪个字段或对象不同? 先核配置与状态
② 过程层	调用顺序、总线交互 或执行轨迹中的对应操作	→	从哪一步开始分叉? 沿操作含义定位
③ 结果层	驱动输出、波形或测量 与项目约定的参考行为	→	结果是否符合判据? 还有哪些观察缺口?

差异回报：操作位置 + 对象 / 字段 + 两边结果 + 当时条件

可选比较对象的机制示意；仅比较约定可比项，不要求全部寄存器或指令轨迹逐项相同。

02

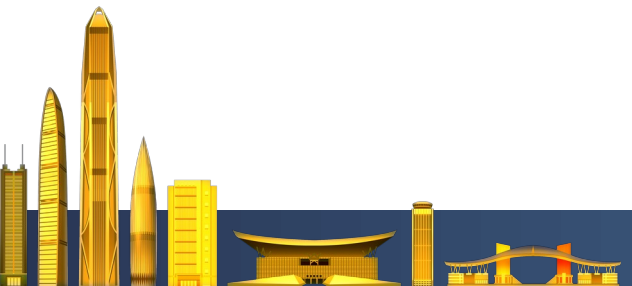
为什么用 Rust

换芯片时复用设备驱动。
AI 改错引脚操作或缓冲区访问时，让编译器及时指出来。

跨平台生态：驱动依赖共同能力，芯片差异留给 HAL 。

错误处理：下层传出原因，上层补齐处理分支并决定策略。

类型与所有权：把引脚模式、缓冲区借用变成编译约束。



同一种 GPIO 操作，厂商接口各不相同

换芯片时，直接依赖厂商 API 的调用代码也要跟着适配

STM32 HAL

```
void HAL_GPIO_WritePin(
    GPIO_TypeDef *GPIOx,
    uint16_t GPIO_Pin,
    GPIO_PinState PinState);
```

```
HAL_GPIO_WritePin(GPIOA,
    GPIO_PIN_5, GPIO_PIN_SET);
```

MSP432 DriverLib

```
void GPIO_setOutputHighOnPin(
    uint_fast8_t selectedPort,
    uint_fast16_t selectedPins);
```

```
GPIO_setOutputHighOnPin(
    GPIO_PORT_P1, GPIO_PIN0);
```

相同意图：把引脚输出置高。
调用者却必须知道每家 SDK 的表达方式。

函数名、参数顺序、引脚标识不同。
直接写进设备驱动，移植时就要逐处改。

共同接口把“需要什么能力”与“芯片怎样实现”分开。

trait：先约定共同能力，再由平台实现

trait 可以理解为接口：先说“要有哪几个方法”，具体怎样操作寄存器，由各芯片自己实现。

接口声明：有哪些操作

```
trait OutputPin {
    fn set_high(&mut self);
    fn set_low(&mut self);
}
```

比如驱动只调用 set_high 。
只要换上的引脚实现了 OutputPin ，
驱动就不必跟着改成另一家 SDK 的函数名。

平台实现：操作自己的硬件

```
impl OutputPin for PA0 {
    fn set_high(&mut self) {
        /* STM32 寄存器操作 */
    }
    // set_low 的实现省略
}
```

```
impl OutputPin for OtherPin {
    fn set_high(&mut self) {
        /* 另一芯片的寄存器操作 */
    }
    // set_low 的实现省略
}
```

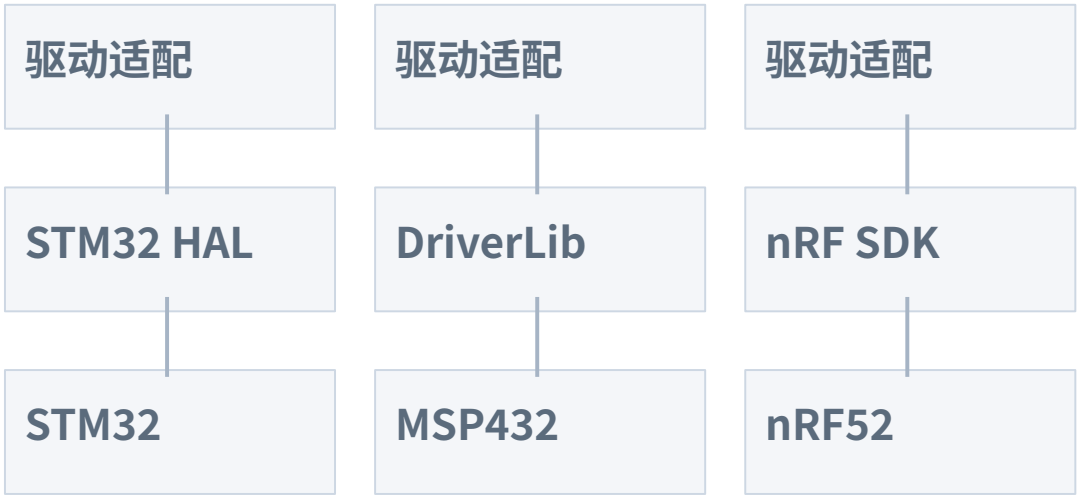
同一方法名，不同硬件实现；芯片差异被放到接口后面。

简化概念 trait ，省略错误返回；不是 embedded-hal 1.0 的完整签名。平台方法体为示意。

embedded-hal：驱动与芯片之间的共同接口

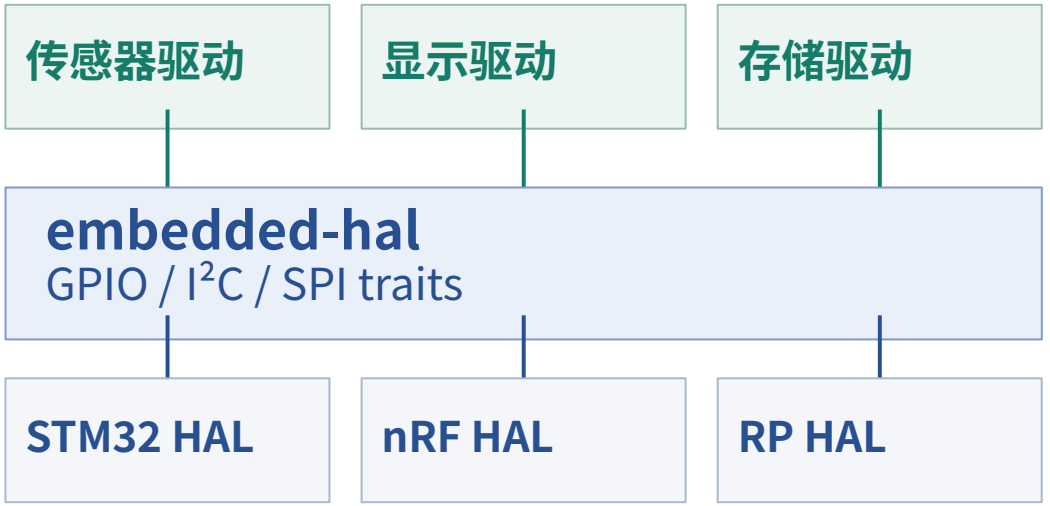
芯片实现保留差异，设备驱动围绕共同能力复用

各自适配厂商接口



左：设备驱动直接适配各厂商接口，
接口变化时，适配与测试也要分别维护。

共同 trait 连接驱动生态



**右：温度转换、寄存器协议、显示命令
写在共同 trait 之上，同一驱动可复用。**

换芯片主要换 HAL 与板级配置；引脚、时钟、总线速度和设备接线仍需正确设置。

理解 trait： C 也可以模拟类似接口

这是辅助类比：手动组织函数指针与上下文，也能表达一组共同操作

C 模拟接口的一个例子

```c
typedef struct {
    void (*set_high)(void* ctx);
    void (*set_low)(void* ctx);
    void* ctx;
} gpio_ops_t;

void toggle_led(gpio_ops_t* gpio) {
    gpio->set_high(gpio->ctx);
    // 延时
    gpio->set_low(gpio->ctx);
}
```

函数指针

调用前要保证函数指针有效，
并把它与正确的上下文绑定。

上下文指针

ctx 指向的对象必须仍有效，
类型也要与回调的预期一致。
void* 本身不会检查这两件事，
配错可能运行时才发现。

Rust 的 trait 与泛型

trait 约定行为，泛型保留具体类型。
能力不满足时，调用在编译阶段被拒绝。

C 也能做好抽象；这里比较的是手动维护的约定，哪些能进入语言的类型检查。

可传播 · C：每层主动检查、返回或包装错误

温度读取经过总线和传感器层。C 中每一层都要检查返回值，再决定继续、返回或包装错误。

① 返回整数错误码

```c
typedef enum {
    ERR_OK = 0, ERR_I2C_BUS,
    ERR_SENSOR_TIMEOUT, ERR_INVALID_DATA
} error_t;
```

② 手动检查返回码，再向上返回

```c
error_t read_temperature_sensor(float *out) {
    uint8_t raw[4];
    error_t err = i2c_read(raw, sizeof raw);
    if (err != ERR_OK) return err;
    *out = convert_to_temperature(raw);
    return ERR_OK;
}
```

I^2C 总线失败
Timeout / Nack 的细节未保留

③ 顶层逐项判断，决定如何处理

```c
float temperature;
error_t err = read_temperature_sensor(
    &temperature);
switch (err) {
case ERR_OK: use(temperature); break;
case ERR_I2C_BUS: report_bus_error(); break;
case ERR_SENSOR_TIMEOUT: retry(); break;
case ERR_INVALID_DATA: discard(); break;
}
```

C 哪里麻烦：每层都要主动处理

每次调用后要检查返回码，再返回或包装。
ERR_I2C_BUS 只保留“总线失败”这一类别。
要保留具体原因、设备和操作，
还要增加字段，并让各层手动传递。

C 可以用结构体保留错误源。这里的麻烦是反复写检查、转换与转发，漏一层就可能丢信息。

可传播 · Rust： 把底层原因装进类型，再向上传递

Rust 把具体原因放进错误类型。？减少逐层 if / return ，设备与操作仍通过 map_err 明确补上。

① 嵌套错误 + From： 保留原始原因

```
enum SensorError { I2cBus(I2cError),
    SensorTimeout, DeviceBusy }
impl From<I2cError> for SensorError {
    fn from(e:I2cError)->Self { Self::I2cBus(e) }
}
```

I²C 总线错误： I2cError::Timeout （超时）

② ？ 代替每次手写 if / return

```
fn read_temperature_sensor()
    -> Result<f32, SensorError> {
    let mut i2c = i2c_init()?;
    let raw = i2c.read(SENSOR_ADDR, 4)?;
    Ok(convert_to_temperature(&raw))
}
```

From 把底层错误包进 I2cBus 。
？成功取值，失败转换后返回。

③ map_err 补设备和操作，顶层取出原因

```
struct ReadError {
    device: &'static str,
    op: &'static str,
    source: SensorError,
}
let result = read_temperature_sensor()
    .map_err(|source| ReadError {
        device: "temp_sensor",
        op: "read", source,
    });
if let Err(err) = result {
    report_context(err.device, err.op);
    match err.source {
        SensorError::I2cBus(cause) =>
            report_bus(cause),
        other => handle_sensor(other),
    }
}
```

顶层看到：设备 temp_sensor / 操作 read / 原因 I²C Timeout （超时）。

可穷尽：新增错误，遗漏的处理分支会显现

顶层明确匹配 source 中每种 SensorError，不使用兜底分支

AI 在底层新增 InvalidData

```
enum SensorError {
    I2cBus(I2cError),
    SensorTimeout,
    DeviceBusy,
    InvalidData,
}
```

AI 改了底层，却忘改顶层 match。
没有红框分支时，匹配不穷尽，
编译器要求补齐遗漏（预期 E0004）。
人或 AI 必须补分支并验证策略。

顶层匹配：红框为必须补上的分支

```
match err.source {
    SensorError::I2cBus(cause) =>
        println!("I2C 错误： {:?}", cause),
    SensorError::SensorTimeout =>
        println!("传感器超时"),
    SensorError::DeviceBusy =>
        println!("温度传感器忙"),
    SensorError::InvalidData =>
        println!("接收到无效数据"),
}
```

右图是补齐后的代码。
编译器指出遗漏，不会自动写好恢复策略。

无 _ 兜底才会暴露新增变体遗漏；？转发层不必逐项匹配。检查分支覆盖，不是运行所有分支。

C HAL： 断言默认关闭，初始化不执行这些参数检查

官方配置模板未启用 USE_FULL_ASSERT 。函数虽然写了 assert_param ，初始化时却不会执行这些参数判断。

调用真实的 HAL_GPIO_Init

```
__HAL_RCC_GPIOA_CLK_ENABLE();
GPIO_InitTypeDef init = {0};
init.Pin  = GPIO_PIN_5;
init.Mode = GPIO_MODE_INPUT;
init.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &init);
```

函数中有参数断言，是否执行取决于宏

```
void HAL_GPIO_Init(GPIO_TypeDef *GPIOx,
                   GPIO_InitTypeDef *GPIO_Init) {
    assert_param(IS_GPIO_ALL_INSTANCE(GPIOx));
    assert_param(IS_GPIO_PIN(GPIO_Init->Pin));
    assert_param(IS_GPIO_MODE(GPIO_Init->Mode));
    /*  后续初始化实现见完整源码附件  */
}
```

同一行 assert_param 的两种展开

```
#ifdef USE_FULL_ASSERT
#define assert_param(expr) \
  ((expr) ? (void)0U : \
   assert_failed((uint8_t *)__FILE__, __LINE__))
#else
#define assert_param(expr) ((void)0U)
#endif
```

默认关闭：参数表达式不求值

IS_GPIO_MODE 等判断不会执行。
代码中有断言，不代表运行时有这道检查。

手动开启：运行到这里才判断参数

条件不满足时调用 assert_failed 。

官方配置模板将 USE_FULL_ASSERT 注释关闭。这里只讨论这些参数断言，不表示初始化函数没有其他逻辑。

PhantomData：用零大小标记表达引脚模式

MODE 是类型信息。 PhantomData<MODE> 本身大小为 0，无需再存一个运行时 mode 字段。

```rust
use core::marker::PhantomData;

pub struct Pin<const P: char,
               const N: u8, MODE> {
    _mode: PhantomData<MODE>,
}
```

同一根 A5，引脚模式对应不同类型

```rust
Pin<'A', 5, Input<PullUp>>
Pin<'A', 5, Output<PushPull>>
```

模式写在类型里

同样是 A5，输入和输出
在 Rust 中是两种不同类型。
PhantomData 关联 MODE 类型，
标记本身不保存 mode 值。

零成本抽象省掉什么

不额外存储运行时模式字段。
通过类型检查的合法调用，
无需逐次判断同一个模式。
实际寄存器读写仍会执行。

类型信息参与检查，标记不增加运行时判断

下一页继续使用这个 Pin 定义：模式由 MODE 表达，方法由对应的 impl 提供。

同一个 Pin：输入模式调用输出方法，编译器拒绝

PhantomData 字段仍在同一个 Pin 定义里。 Output 专属 impl 提供 set_high ， Input 类型没有这个方法。

① 沿用相同的 Pin 定义

```
pub struct Pin<const P: char, const N: u8, MODE> {
    _mode: PhantomData<MODE>,
}
```

② 只给 Output 模式提供 set_high

```
impl<const P: char, const N: u8, M>
    Pin<P, N, Output<M>> {
    pub fn set_high(&mut self) { /* 写寄存器 */ }
}
```

③ AI 把调用处的类型改成 Input

```
fn update(pin: &mut Pin<'A', 5, Input<PullUp>>) {
    pin.set_high();  // 预期错误 E0599
}
```

同一字段，记录模式类型

MODE 可以是 Input 或 Output。
_mode 仍是零大小标记，
不保存运行时的模式值。

方法由 impl 的类型限定

set_high 只在 Output 模式上定义。
编译器查找 Input 类型的方法时，
找不到 set_high ，所以拒绝调用。

PhantomData 保留类型关联；真正拒绝错误调用的是方法的类型约束。

预期诊断，未实际编译。只拦已编码的错模式；物理引脚选择和接线仍需验证。

GPIO 源码与汇编：相同操作， C 示例多一段参数判断

C 主动检查指针与编号。 Rust 使用正确构造的 A5 输出引脚，合法调用无需重复这些参数判断。

C： 运行时参数与显式检查

```c
void toggle(volatile uint32_t *odr, uint8_t pin) {
    if (odr == NULL) return;
    if (pin > 15) return;
    *odr ^= 1u << pin;
}
```

对应汇编示意

```
cmp r0, #0
beq .done
cmp r1, #15
bhi .done
movs r3, #1
lsls r3, r3, r1
ldr r2, [r0]
eor r2, r2, r3
str r2, [r0]
.done: bx lr
```

判断失败直接返回
跳过下方寄存器操作

计算 1 << pin
本次 pin = 5 ，结果为 32

读取 ODR
翻转目标位
写回 ODR

Rust： A5 输出引脚的类型接口

```rust
fn toggle_led(
    led: &mut Pin<'A', 5, Output<PushPull>>
) {
    led.toggle();
}
```

对应汇编示意

类型接口已限定合法引脚。
无需重复 NULL 与编号判断。

```
movs r3, #32
ldr r2, [r0]
eor r2, r2, r3
str r2, [r0]
.done: bx lr
```

读取 ODR
翻转目标位
写回 ODR

两侧读改写指令相同。汇编为示意，非实测； r0 为 ODR 地址， r3 为位掩码。条件已知时 C 检查可能被优化。

生命周期：编译器检查引用能否继续使用

生命周期是编译器追踪引用有效性的机制。
引用不能超过它所指向数据的有效期，标注生命周期不会延长数据的寿命。

C 通常由程序员维护指针何时仍有效，避免悬垂指针或访问已释放的内存。
Rust 的安全 API 通过借用和生命周期约束，在编译阶段阻止相关误用。

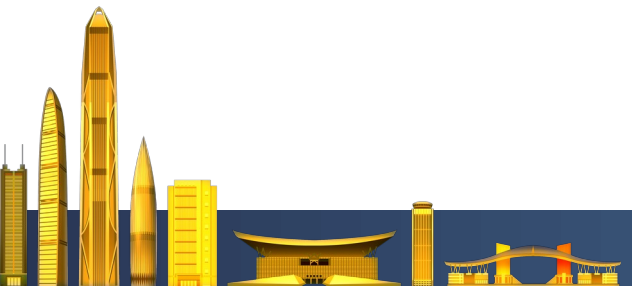
例子：返回引用，不复制原数据

```
fn first<'a>(data: &'a [u8]) -> &'a u8 {
    &data[0]
}
```

两处 'a 关联输入与返回值。
返回引用仍依赖原数据，
不能在原数据失效后继续使用。

放到 DMA 上，还需要安全 API 管住硬件和 buffer

编译器检查的是代码中的借用和所有权。
DMA 的完成检查、停止和 buffer 归还，需要由 HAL 的相应流程处理。



C DMA：还没传完，就修改 buffer 会怎样?

buffer 指向要发送的数据， size 是字节数。启动函数返回时， DMA 可能还在读取。

① 启动函数内部做了什么

```
void start_dma_transfer(uint8_t *buffer,
                        size_t size) {
    DMA_Config(DMA1, buffer, size);
    DMA_Start(DMA1);
}
```

② AI 插入一行修改，硬件可能还在读

```
uint8_t buffer[64] = {0};
start_dma_transfer(buffer, 64);
buffer[0] = 99;  // AI 新增
while (DMA_GetStatus(DMA1) != DMA_COMPLETE) {
    /* 等待完成 */
}
```

参数说明了传输范围

Config 设置地址和字节数，
Start 启动传输。
启动函数返回时，
DMA 可能仍未读完。

中途写入可能改变传输结果

DMA 还没读到第一个字节时，
CPU 若先把它改成 99，
发出的数据就可能改变。
这组指针参数没有告诉编译器：
此时不能修改 buffer。

若 DMA 还未读到该字节，且写入对它可见，传输可能被污染； C 也可通过更严格封装管理此约定。

Rust DMA： 把传输期间的 buffer 访问交给类型检查

AI 可以继续修改发送逻辑，但传输对象持有 buffer 时，不能绕过它直接改这块内存。

```
let buffer = cortex_m::singleton!(
    : [u8; 64] = [1; 64]).unwrap();
let mut transfer =
    Transfer::<_, 0, _, MemoryToPeripheral, _>
        ::init_memory_to_peripheral(
            stream, tx, buffer, None,
            DmaConfig::default().memory_increment(true),
        );
transfer.start(|_| {});
transfer.wait();
let (_, _, buffer, _) = transfer.release();
buffer[0] = 99;
```

① 传输对象持有 buffer

静态缓冲区的独占引用
实际存入 transfer 。
调用者暂时交出访问权，
不能再用原变量直接修改。

② 等待 DMA 搬运完成

transfer.wait() 确认
这一轮 DMA 搬运已完成。

③ 取回 buffer ，再修改

release() 停流并归还资源，
从返回值取得 buffer 后再改。

AI 漏掉“传输期间不能改”的约定，编译器也会检查

基于社区 stm32f4xx-hal v0.23.0 的发送 API； 静态独占 buffer ，板级配置与导入省略。

AI 在传输途中加了一次写入： C 与 Rust 的不同结果

同样新增 buffer[0] = 99： DMA 可能还在读取，代码却准备改动要发送的数据。

C： 指针接口没有表达独占使用

```
start_dma_transfer(buffer, 64);
buffer[0] = 99;  // AI 新增
while (DMA_GetStatus(DMA1)
       != DMA_COMPLETE) {
    /* 等待完成 */
}
```

这组接口允许编译，传输可能受影响

如果 DMA 还未读取这个字节，
CPU 的写入就可能改变后续发送的数据。

Rust： transfer 仍持有独占 buffer

```
transfer.start(|_| {});
buffer[0] = 99;  // AI 新增
transfer.wait();
let (_, _, buffer, _) =
    transfer.release();
```

编译拒绝： buffer 尚未归还

独占 buffer 已经交给 transfer。
AI 新增的提前写入，过不了所有权检查。

修复：完成搬运，取回 buffer 后再修改

```
transfer.wait();
let (_, _, buffer, _) = transfer.release();  buffer[0] = 99;
```

把 AI 容易遗漏的资源使用约束，变成编译期检查。

03

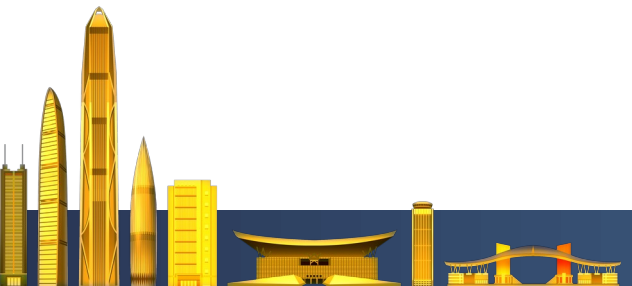
CoHDL：让器件知识留在设计里

AI 读完 Datasheet，把核对过的条件写进模型，后面继续用。

留下知识：器件允许什么、应该怎样连接

复用电路：下一次沿用模型和子电路

一起检查：人和 AI 逐块核对设计



CoHDL 的设计入口

把手册里核对过的要求写进模型，让下一次修改仍能检查

原理图编辑入口

放置器件符号，画连接
由编辑器维护电路关系

DSL 编程入口

用代码声明器件和引脚
写明连接、规格与复用电路
修改后重新检查已有规则

网表与物料

网表：哪些器件引脚电气相连
物料：使用哪种器件与封装

EDA：继续摆放、布线与规则检查

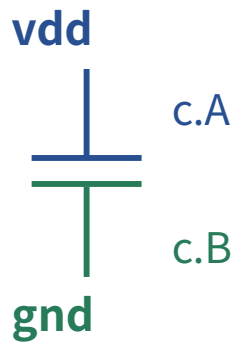
Explorer：人和 AI 逐块审阅模型里的器件与连接

EDA 也可通过第三方 MCP 供 AI 调用，已有 ERC/DRC 与复用能力； DSL 生成不免除物理设计和实物验证。

以 100nF 去耦电容为例：连接关系、服务对象和复用方式都写进代码

net 写连接， bypass 把去耦意图留给 AI ， fn 复用整段电路

```
pub fn decoupling_100n(vdd: Pin, gnd: Pin) {
    #[bypass(vdd, 100nF)]
    inst c: C_100n_16V_X7R_0402
    net _: vdd, c.A
    net _: gnd, c.B
}
```



c · 100nF

① 代码中的实例 c

② 两条 net 对应上下两段连接

net 只说明电气相连

bypass 记录电容服务的器件、引脚和容量

这些关系随源码进入 AI 上下文，同一电源网有多个器件时，也能查到电容服务谁。

fn 在编译时展开：调用两次，就得到两颗独立电容

器件、连接与 bypass 标记一起复用，减少复制遗漏；它不是运行时函数。

真实库函数节选。用途标记不保证布局距离或去耦效果，展开后的电路仍按已有规则检查。

Datasheet 知识如何成为可复用的语言约束

把手册条件写入器件模型，再与网络电压核对；需求变化后，同一规则继续检查。

① 手册给出工程条件

推荐输入工作范围
2.5–6.0 V

绝对最大值 6.5 V
不能用作正常工作上限

本例先编码 6 V 上限。
来源和适用条件一起保留，
下个项目无需重新猜测。

② 器件库保存可检查的条件

```
#[doc("docs/diodes-ap2112.pdf")]
pub device AP2112K_3V3 {
  pins { /* 引脚声明略 */ }
  spec {
    voltage_rating: 6V
  }
}
```

③ 电源需求变化后重新检查

假设输入由 5V 升到 6.2V，
网络电压同步更新。
原器件模型上限仍是 6V。

D001： 6.2 V > 6 V
网络电压超出器件模型上限

调整实际供电或更换
合适器件，再检查。

手册条件跟着模型走，需求变化后仍可逐次核对

输入电压与器件上限一起留在工程里，换一轮 AI 也沿用同一份检查。

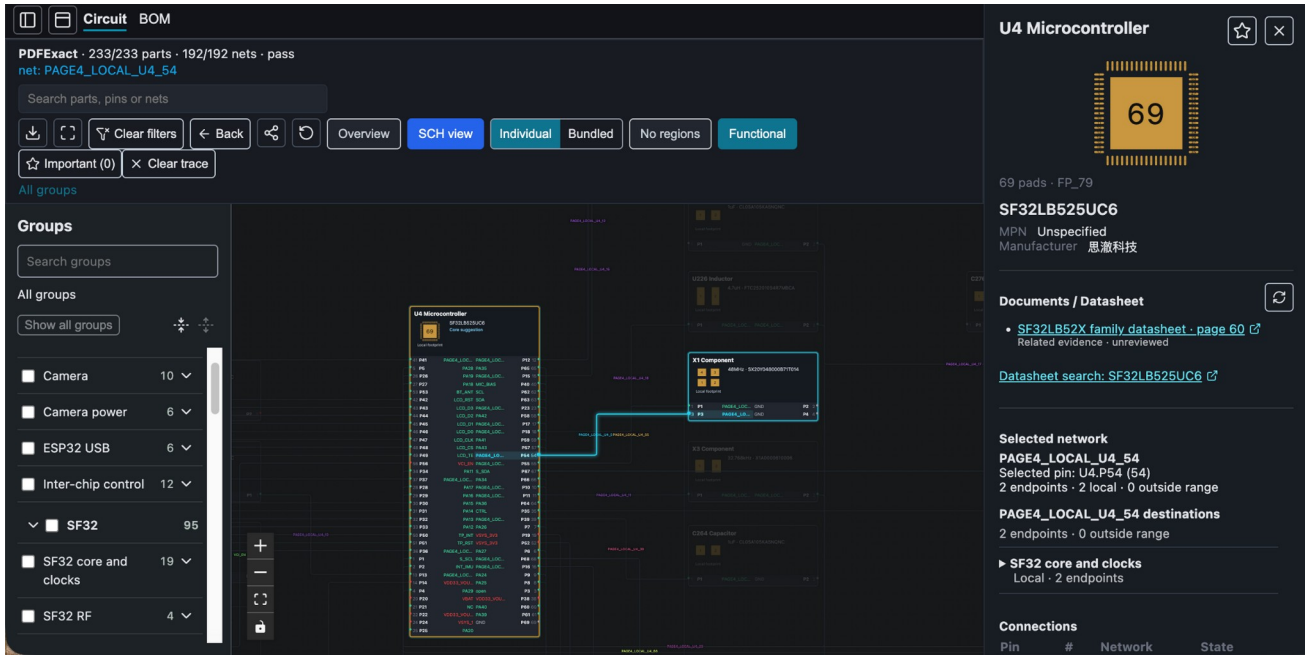
示例需求变更； 6.2 V 越界已有 CLI 验证。仅检查显式网电压与已编码上限，其他电气条件仍需验证。

Explorer： 人和 AI 逐块检查电路

CoHDL / Explorer 分组审阅

按模块查看器件与连接，结合资料核对，再检查跨模块关系

全图：按问题分组，保留全局与资料入口



人和 AI 依次审阅功能模块
结合器件资料核对设计

局部放大： U4.P54 的连接



高亮网络连接 U4.P54 与 X1.P3
局部检查后，再返回全局核对

可跳过手工绘制完整原理图；电气、布局与实物检查仍需完成。

CoHDL → KiCad： 生成设计起点，继续布局与验证

网表给出连接关系，板文件承载封装与已声明放置；物理设计仍要继续

```
cohdl build --emit kicad_pcb
```

① 生成电路产物

KiCad 板文件 .kicad_pcb
网表、 BOM 与封装
带网络与已声明放置

② 在 KiCad 完成布局布线

由人或具备相应能力的 AI 工具继续
摆放器件、走线、铺铜并配置 PCB 规则

③ DRC 与工程审查 → 制造 → 实物验证

生成板文件，不等于已经完成布线或通过产品验收

可选输出为 KiCad10 板文件。 CoHDL 不在此完成布线、铺铜或全部板级规则； AI 辅助布局结果同样需要检查。

04

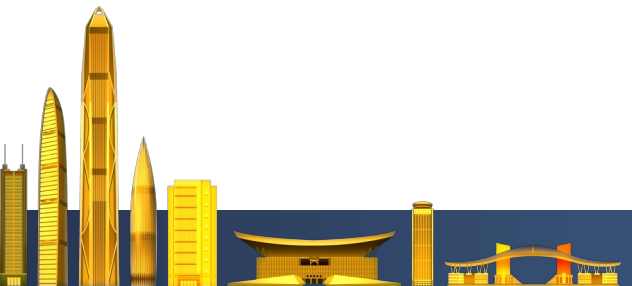
Kit：可重复使用的 AI 硬件工位

一次配置工具接入，让后续任务复用设备能力。

人完成接线，明确目标与操作边界

AI 调用工具，连续调试与验证

多套独立工位，支持任务并行



一个 Kit，就是一个 AI 工位

Kit / 聚合研发工位

接好设备后， AI 调用工具烧录、读取串口、采集波形；后续任务继续复用

AI / 项目实验
这次要测什么、怎样算通过

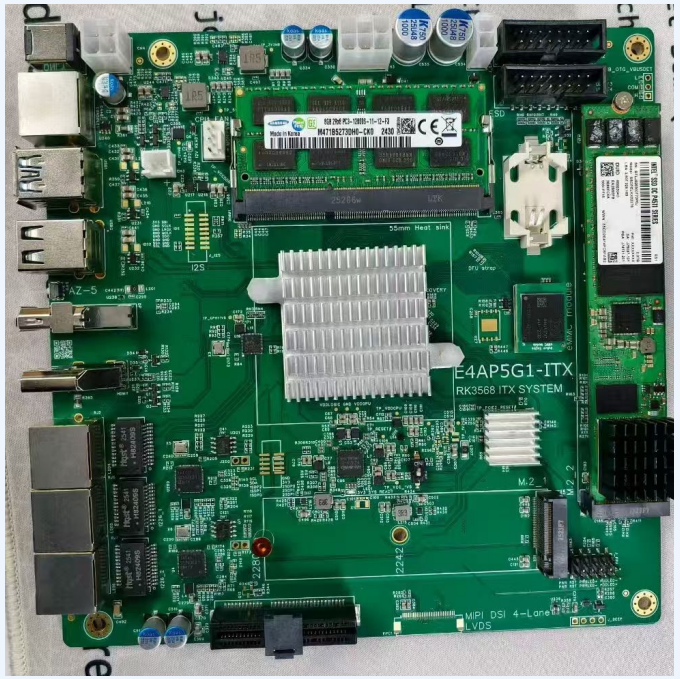
蓝：操作要求　紫：观测回传

调试器 / 串口
烧录、调试，保留原始收发

逻辑分析仪
数字信号与通信时序

示波器
模拟电压、波形与边沿

自研 RK3568 Linux 主机
运行工具，取得设备结果



统一回传实验记录
测了哪块板、在什么条件下

LCR 表 / 万用表
电感电容、阻值、电压电流

摄像头
外观、连接与显示画面

红外热成像分析仪
温度分布与热点观察

…… 按任务扩展更多仪器

配置工具接入后，多个任务复用同一工位能力

仪器按实际适配接入；操作与观测保留设备、固件、接线和采样条件。

Kit 工具：执行动作，返回原始证据

按问题调用工具，保留原始结果，再决定补测或修复

验收问题

规定场景下，板上通信是否正常?

→ **实验执行**

通过 CLI / MCP 调用工具
对已接线的板卡执行实验

→ **每次实验携带身份**

设备 ID 、固件版本、执行步骤
接线、通道与采样参数

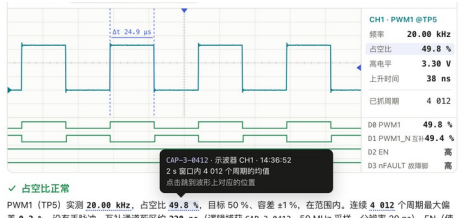
要回答什么	用什么观测	回传原始事实
① 固件身份与应答	调试器 + 串口	身份记录 + 原始收发
② 通信时序	逻辑分析仪	数字波形 + 时间关系
③ 电压与边沿	示波器	模拟波形 + 采样参数

项目判据比较

两次测试条件一样吗?
证据还缺什么?
结果达到项目要求了吗?

缺证据：补观测
未满足：修复后复测

既有 Web 波形记录



仪器画面示例；数值不作为本页测量结论

AI 根据实际读数判断：继续补测，还是修改后重测

按验收问题选择观测手段；具体仪器与接线依实际配置，不能仅凭仪器接入判断测量有效。

接线完成后， AI 连续调试与验证

人完成接线与边界确认， AI 在工具支持范围内连续执行

① 人准备工位

接线与设备确认
目标和验收条件
允许操作的范围

② AI 修改与运行

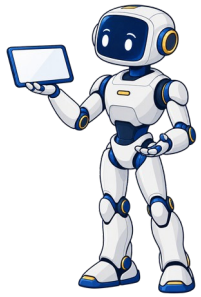
修改、构建固件
烧录并运行
调用调试与测量工具

③ AI 分析与修复

比较目标与原始观测
定位差异并修复
按同一条件复测

④ 工位与证据持续复用

每轮保留代码、固件版本、实验条件和原始结果
独立 Kit 可并行，共享设备先获锁



“接着调，直到满足当前判据”

需要改接线、越界或关键决策时交人处理

自主调试方式示意，限于已接入工具与授权范围；不代表本轮已完成无人值守端到端实测。

05

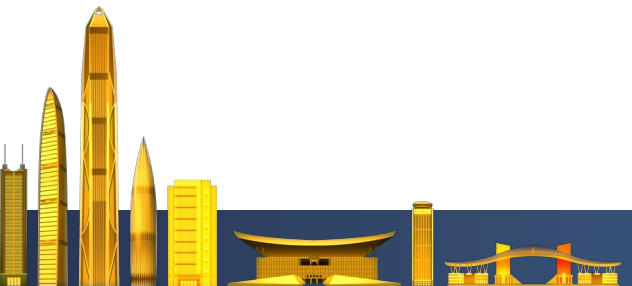
持续推进的研发工厂

围绕产品目标，组织模块研发、整机验证与持续迭代。

模块分层：模型、 Harness 、 Factory 与 Connect

研发接续：项目文档、多工位与方案探索

独立 QA ： 贯穿研发，也可单独检查其他项目



Aragorn 系统： Harness over Harness

下层执行当前任务，上层安排下一步，并检查任务是否真正完成

上层职责

人 / Connect
持续需求与关键决定
查看进展、证据与结果

→

Factory 编排与 QA 验证

厂长 / Foreman
拆分任务，检查进展
决定下一步怎么做

Worker
完成分配的工作并返回结果

独立 QA
按标准检查产物与证据

项目需求、设计与任务约定贯穿执行， QA 检查当前产物与证据

同一种角色可以接不同引擎，按任务选择适合的执行方式

Cursor	Codex	Claude Code	Pi	OpenCode
运行一个 Agent	运行一个 Agent	运行一个 Agent	运行一个 Agent	运行一个 Agent

引擎 harness： 文件读写、命令工具、模型 / 工具循环

模型配置 不同模型 / provider / effort； 按任务配置

模型按任务与预算选择；具体引擎能力、权限与隔离由适配和配置决定

工厂设计：高能力模型推进，成本友好模型执行

设计原则：全局判断投入更强能力，边界清晰的细节任务按风险配置模型

人 / Connect

目标、边界与关键取舍

高能力模型：交互、规划与厂长推进

澄清需求、拆任务、检查进展
决定继续执行、调整方案或升级处理

失败 / 高风险
交厂长升级

屏幕任务 Agent

边界清楚的实现 / 检查
成本友好、 Flash 级模型

供电任务 Agent

边界清楚的实现 / 检查
成本友好、 Flash 级模型

连接任务 Agent

边界清楚的实现 / 检查
成本友好、 Flash 级模型

独立 QA

核对产物、证据与验收
模型便宜不降低标准

符合条件再推进

产物、执行记录与未完成项持续回报

模型分工理念，非固定默认配置；高风险或复杂任务按验证结果升级

DevSwarm： 组织多个 AI 分工与竞赛

Aragorn / DevSwarm

多 AI 可以分工协作，也可以让不同模型独立完成同一任务，按共同证据比较

① 不同任务分工

拆分工作与角色

同一项目内，明确各任务的输入与产出

- **屏幕任务**
  AI 1： 显示实现
- **供电任务**
  AI 2： 模式验证
- **连接任务**
  AI 3： 通信实现

汇总与集成

汇总补丁与证据，处理接口冲突和缺口

② 不同模型，同题竞赛

完全相同的任务、输入与验收条件

任务一致，利用不同模型的能力差异

- **Agent A**
  模型 A
  完成同一任务
- **Agent B**
  模型 B
  完成同一任务
- **Agent C**
  模型 C
  完成同一任务

共同 QA： 比较结果与执行证据

按同一条件选取可用成果，集成后再验证

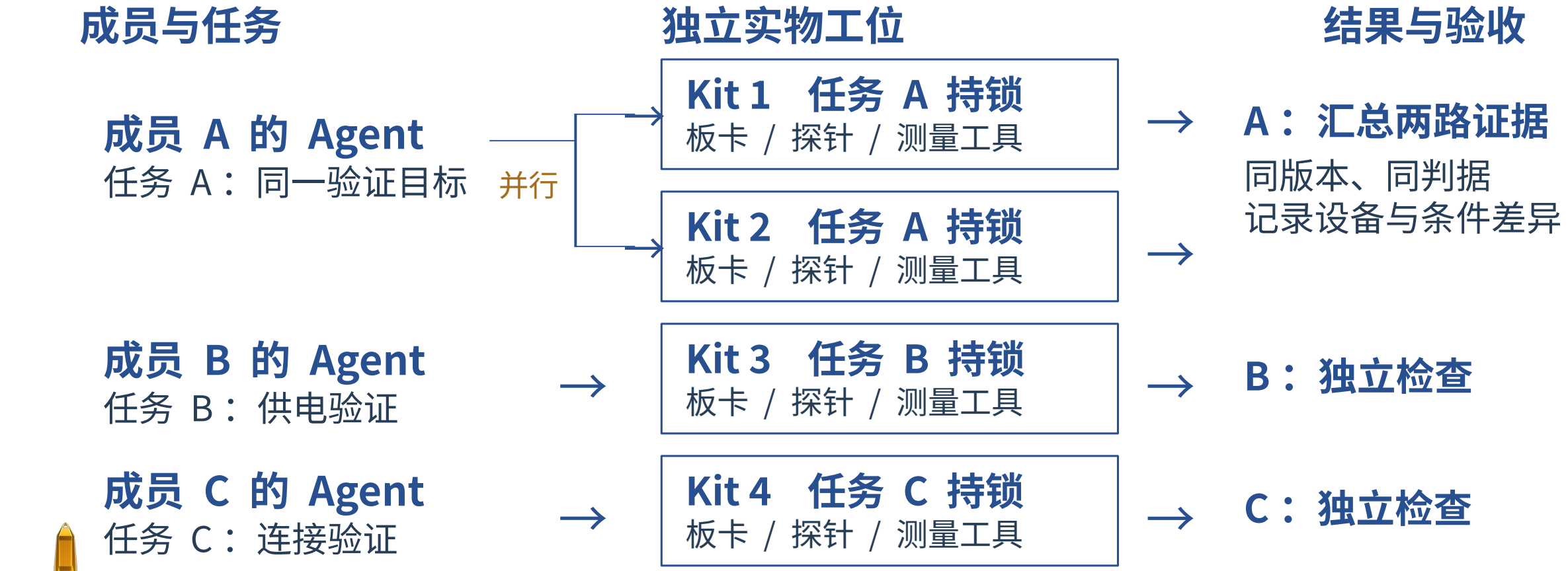
开发结果汇合后

集成验证、独立评审与产品验收

不同模型各有能力边界；投入按任务、预算与验证结果配置，不要求每路都用最高成本模型。

四个实物工位：同任务并行测量，多任务同时推进

任务 A 使用两个独立 Kit 测同一对象；其他任务使用各自工位



各任务提交对应 QA；缺口回到所属任务继续修复

共享同一物理资源时，持锁者操作，其他请求等待

QA： 贯穿研发，也可以独立使用

开发中、交付前或单独检查仓库，都使用同一套验收标准与证据规则

开发过程中

任务执行后的持续检查

里程碑与完工

检查目标覆盖与整体结果

任意仓库单独使用

独立评审、检查与报告

输入

已确认的标准与目标
当前产物和版本
真实执行的检查证据

共享 QA 内核

运行检查，保存证据
真实运行检查，保存原始输出

核对证据是否适用
版本与范围一致吗？评审独立吗？

按同一标准判断
入口可以不同，通过条件不能变

可追查的结论

满足当前范围的条件
或指出具体缺口
并给出下一步修复方向

缺口回到实现与验证，修复后按当前版本重新检查

项目看板与架构资料：人和 AI 共用的研发上下文

既能看清项目运行到哪里，也能理解整体设计和每项工作的依据

人

看整体架构
查进展与阻塞
作关键决定

⇄

整体架构与项目资料

需求与边界　架构与接口
设计选择　路线图与里程碑
代码版本　检查记录与原始证据

⇄

AI

读取设计依据
核对任务和证据
执行后同步更新

共享项目看板

各项工作更新后，人和 AI 都能继续读取

当前进展

做到哪里
哪些已经完成

阻塞与依赖

卡在哪里
等待什么条件

证据与版本

结论来自什么
对应哪个版本

下一步

谁继续执行
先处理哪个缺口

需求变化同步到设计与任务；状态更新附上可追溯的依据

长任务：厂长依据进展，持续选择下一步

共享看板、架构资料与真实结果，为灵活判断提供依据

持续读取

产品目标与架构
当前进展和阻塞
新增产物与证据
投入、风险与剩余工作

→

厂长
综合判断
选择下一步行动

→

继续
方向正确，已有有效进展

重试 / 改派
补上下文、换方法或模型

调整计划
重排依赖，重拆关键任务

交人决定
涉及关键取舍或权限边界

执行结果回到看板，再判断

模型分工 厂长采用更强模型； Worker 按任务选用成本友好模型

判断随结果、阻塞和目标变化触发，不依赖固定轮换时长

人的工程经验，形成 AI 探索的路线假设

Aragorn / Path Matrix 经验

先用经验提出值得试的方案，再让 AI 实现和验证，留下选与不选的理由

经验告诉我们先试什么

“这条路适合什么条件，
当时为什么没有选？”



人给判断与假设
AI 补候选与反例

用实际尝试扩展经验，而不只凭直觉猜



路线 A

实现候选，取得可重查的证据

保留成功条件与当前版本



路线 B

改变一个关键假设再验证

留下收益、代价与适用边界



路线 C

检验反例与未满足条件

失败或未完成，也有接续价值

Path Matrix 留下：试了什么、结果如何、为什么这样选

条件变化后，已有经验进入下一轮假设与实验

已有路线记录与证据机制；更自主的候选生成、实验分配和组合探索是演进方向。

Path Matrix： 各个决策点的多种实现

Aragorn / Path Matrix 候选矩阵

人的经验提出候选， AI 在资源边界内逐项实现，按同一条件验证与比较

产品方案示例

产品决策点	候选实现 1	候选实现 2	候选实现 3
屏幕刷新	D1 定时全屏刷新 实现 · 检查 · 记录证据	D2 按需全屏刷新 实现 · 检查 · 记录证据	D3 局部区域刷新 实现 · 检查 · 记录证据
供电管理	P1 固定运行模式 实现 · 检查 · 记录证据	P2 空闲进入低功耗 实现 · 检查 · 记录证据	P3 按外设分域控制 实现 · 检查 · 记录证据
连接方式	C1 周期轮询 实现 · 检查 · 记录证据	C2 事件通知 实现 · 检查 · 记录证据	C3 事件与周期结合 实现 · 检查 · 记录证据

每一行按共同条件比较

是否满足要求、资源代价、适用边界、未解决问题

选取当前可用实现，组合后再次验证；其余候选与取舍原因继续留存。

Path Matrix： 执行记录成为可复用的经验

Aragorn / Path Matrix 执行记录

每个候选都保存代码、版本与证据，以及留下或舍弃它的原因

路线档案

同一目标与比较条件下的探索记录

路径 / 假设	当前版本与证据	未选原因 / 缺口	重开条件
D2 全屏 按需刷新	代码与提交版本 检查记录 / 引用	保留的依据 适用边界与缺口	交互需求改变 重新检查适用性
D3 局部 局部区域刷新	候选代码与版本 实验记录 / 待验证	未选的理由 当前验证缺口	目标或资源改变 重新申请执行

接回同一条路线

1 对照新条件检索旧档案

2 核对假设、版本与证据缺口

3 重新准入，执行必要验证

4 与其他候选按共同条件比较

旧证据提供线索，当前结论重新取得

记录结构示意，不代表候选已取得实测结果。

未选路线继续保留；重新接回后，仍固定当前版本、更新证据与取舍原因。

需求变化后，重新查看过去的候选

手表产品示例：交互需求变化，让原来未选的实现重新具有参考价值

原有需求

内容变化少
先满足基本显示与响应

D2 按需全屏刷新

当前选择的原因写入档案
D3 局部刷新暂不采用
保留实现和当时的验证缺口

需求发生变化

交互频率提高
小区域内容经常更新

重新查看 D3

原先的取舍是否仍适用?
复用已有代码与记录
明确还缺哪些当前证据

按当前条件重新比较

D2 与 D3 使用相同输入
重新验证显示正确性
比较响应、功耗和实现代价

依据新证据决定取舍

若选择 D3，记录当前版本
并验证与供电、连接的组合

需求情境示意，不预设 D3 一定更优，也不把历史证据当作本轮通过。

经验的价值：知道试过什么、当时为什么没选，以及什么时候值得再试。

模型能力提高后：更充分地探索候选空间

Aragorn / Path Matrix 自主探索

已有路线与证据基础；更自主的假设生成、实验分配和组合探索是演进方向

① 共同目标：产品响应与续航　范围：已定义候选、预算、工位、硬件边界与验收条件

② AI 提出假设与小实验

先把经验写成可检验的假设
AI 补充值得试的方案

再按预算与工位安排：
实现、检查与实物实验

③ 假设得到这些反馈

A： 保留现有硬件
检查软件策略的收益边界
B： 增加供电控制
检查改板代价与恢复行为

测到一个现象，还要继续查明原因

根据结果，决定下一轮试什么

④ 提出新组合与下一轮实验

例如组合供电控制与软件调度

先检查两个部分能否分别成立
再测试组合后的响应、功耗
以及从低功耗状态恢复的行为

⑤ 按同一外部目标验证

满足适用判据 → 可选成果
未满足 → 修正假设 / 候选 / 实验

⑥ 方案与经验储备

代码、证据、优缺点、适用条件
成功 / 未选 / 未完成 / 反例都留痕

新的证据回到下一轮
规划、假设与实验

产品探索设计示意，非实测结果；系统覆盖给定候选，不等于穷尽无限空间或保证全局最优。

Connect： 多个 Agent 的统一入口

工厂持续推进研发，人不必一直守在电脑前，也不必跟着某个 Agent 切换入口

人

查看进展与证据
补充需求
确认关键决定

Connect 统一入口

同一个项目与任务视图
汇集各 Agent 的执行状态
把问题、证据和决定接起来

持续运行的研发工厂

- 厂长：组织与推进
- Worker： 执行与回报
- QA： 核对证据与判定

不同 Agent / Harness 与模型按任务配置

离开电脑时工厂继续推进；需要人决定时，通过统一入口接续。

Connect： 自建 Web 工作台，把任务和观测放在一起

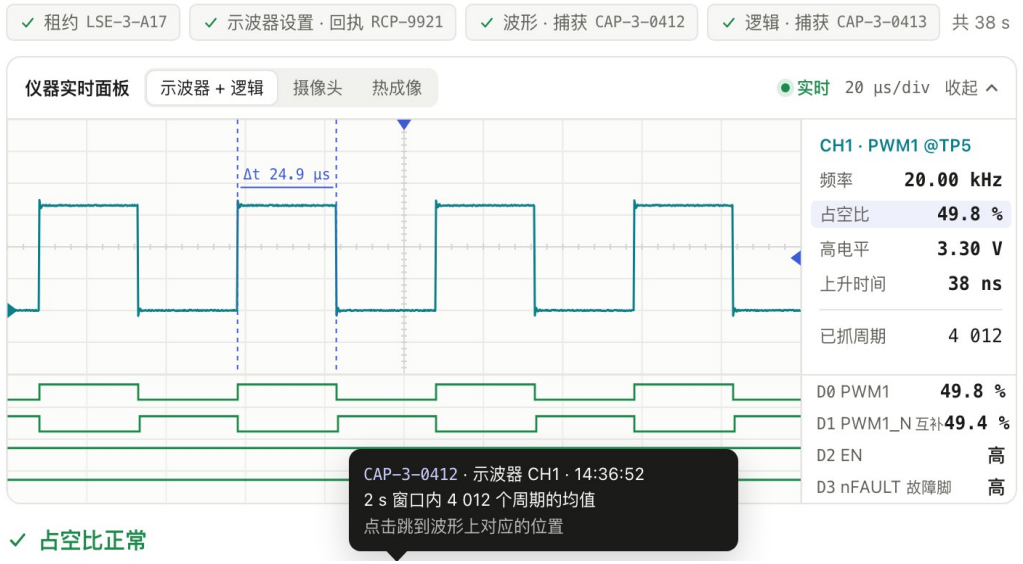
Aragorn / Connect Web 工作台

在同一个工作台查看任务、实验过程与工位，并把问题交回人

全图：任务、待办与工位状态



局部放大：波形与采集回执



操作条件、波形 / 逻辑捕获与回执编号
一起回到任务，支持复核与下一步决策

自建 Web 和 IM 都可作为 Connect 的协作入口

电脑端实图：机器、 Session 与同轮进展回报

Aragorn / Connect 电脑端

选机器、进入会话，围绕同一任务持续收回报与给反馈



执行发生在远端机器，界面连接人与当前任务

1 Group： 选机器

进入承载项目的执行机器

2 Topic： 选会话

在任务上下文中继续协作

3 看进展，提反馈

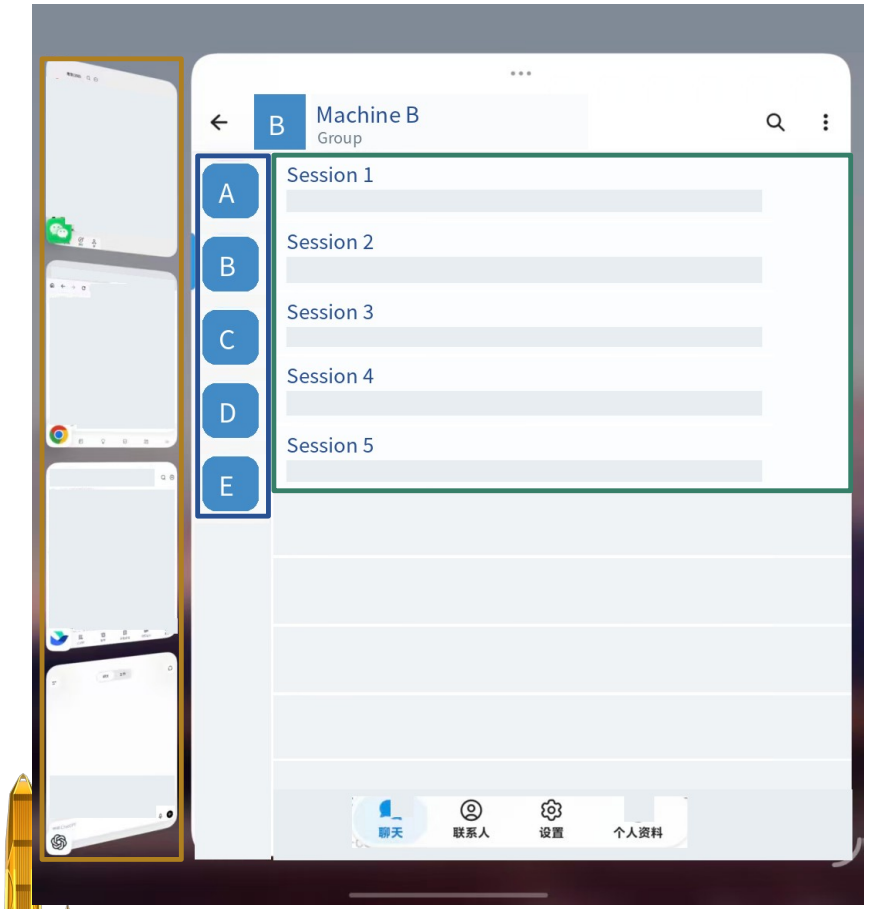
4 用状态图对齐理解

文字与图片一起回报
任务仍按验收条件完成

手机端实图：选机器、进会话、继续反馈

离开工位后，仍能回到正在推进的任务

手机端实际入口



1 手机也是协作入口

查看任务进展，带着上下文继续反馈

2 Group： 选择机器

选择正在执行项目的远端机器

3 Topic： 进入 Session

回到对应任务，继续提问和给反馈

手机上继续协作，执行仍在连接的机器上

现场接线、物料操作与必要的人工判断仍需配合

让 AI 持续工作的工程依据

每一层都给下一轮修改提供可定位的反馈，而不是只相信一次生成结果

01	软件约束	Rust 接口与类型	检查调用条件、资源冲突与遗漏分支
02	电路条件	CoHDL 工程模型	修改设计后，重新检查已写明的条件
03	真实观测	Kit 仪器与设备	把设备实际表现交给 AI 判断
04	持续组织	Aragorn 任务与验收	知道做到哪一步、还缺什么、谁继续

人定义适用的质量判据， AI 持续把实现和证据补齐

静态约束、模型检查与实物验证各有覆盖范围，组合使用仍需项目与产品验收。

模型继续进步，公司的研发方式也持续演进

公司围绕产品目标组织任务，用真实证据检查进展与交付质量

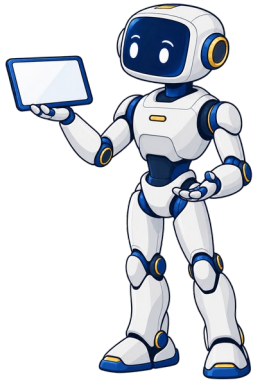


把握产品方向与关键取舍

理解用户、确定产品目标
安排关键资源与架构取舍
协调合作、判断交付质量

持续推进项目任务

持续实现、实验与修复
让进展和证据保持可见
按产品要求完成验证



今天的一些 Harness，未来可能被简化、替代，甚至不再需要

我们追求的是

在每个阶段，找到公司最高效推进项目与产品的方式

Harness 会变，公司的任务组织与研发方式也会随之调整

谢谢

欢迎交流

